Software Design Document (SDD)

ReForge

Tagline: The Self-Healing RAG Pipeline

1. Executive Summary

ReForge is a production-oriented Retrieval-Augmented Generation (RAG) system that uses a LangGraph-based multi-agent workflow to detect hallucinations, critique its own answers, rewrite queries, and retry retrieval before responding.

2. Problem Statement

Conventional RAG systems retrieve documents once and immediately generate answers, which can lead to hallucinations if retrieval quality is poor. ReForge introduces a self-healing feedback loop.

3. Objectives

- Build a reliable AI agent.
- Reduce hallucinations.
- Demonstrate agentic workflows.
- Provide explainable, grounded answers.

4. Functional Requirements

User query, document ingestion, semantic retrieval, grounded generation, critic evaluation, query rewriting, retry loop, graceful failure, source citations.

5. Non-Functional Requirements

Scalability, modularity, maintainability, observability, explainability, low latency, extensibility.

6. High-Level Architecture

Frontend (React + Next.js) → FastAPI Backend → LangGraph Orchestrator → Retrieval Agent → Generator Agent → Critic Agent → Decision Agent → Rewrite Agent → ChromaDB → Gemini LLM.

7. Workflow

User → Retrieve → Generate → Critic → Decision. If rejected, rewrite query and repeat until max attempts, otherwise return a graceful failure.

8. LangGraph State

State fields: question, rewritten_question, retrieved_docs, similarity_scores, answer, grounded, confidence, critic_feedback, attempts, max_attempts, final_answer.

9. Agent Design

Retrieval Agent, Generator Agent, Critic Agent, Rewrite Agent, Decision Agent. Each agent has a single responsibility and communicates via shared graph state.

10. Technology Stack

Frontend: React + Next.js
Backend: Python + FastAPI
Agent Framework: LangGraph
RAG: LangChain
LLM: Gemini 2.5 Flash
Embeddings: all-MiniLM-L6-v2
Vector DB: ChromaDB
Monitoring: LangSmith
Deployment: Vercel + Render/Railway

11. API Design

POST /upload-documents
POST /chat
GET /history
GET /health
DELETE /documents/{id}

12. Folder Structure

frontend/, backend/, agents/, graph/, prompts/, vectorstore/, api/, utils/, tests/, docs/.

13. UI Pages

Home, Chat, Upload Documents, Knowledge Base, Execution Trace, Settings.

14. Database Design

Persistent ChromaDB collections with metadata for source, page, chunk id, embedding id.

15. Security

Environment variables, API authentication, file validation, prompt injection safeguards.

16. Testing

Unit tests, integration tests, retrieval accuracy tests, end-to-end workflow tests.

17. Deployment

Frontend on Vercel. Backend on Render/Railway. Chroma persistent storage. GitHub Actions for CI/CD (optional).

18. Future Enhancements

Hybrid search, reranking, multi-modal RAG, OCR, conversation memory, analytics dashboard, human-in-the-loop, enterprise connectors.

19. Development Roadmap

Phase 1: Core RAG.
Phase 2: Self-healing loop.
Phase 3: Explainability and monitoring.
Phase 4: Advanced retrieval.
Phase 5: Production deployment.

20. Conclusion

ReForge showcases modern AI engineering by combining retrieval, generation, self-critique, and iterative reasoning into a reliable, explainable agentic system.